
Tecnicatura Universitaria en Desarrollo Web

Laboratorio Web 1

Profesor: Jorge Ceferino Valdez
Unidad Académica Caleta Olivia
Universidad Nacional de la Patagonia Austral

Guía paso a paso — Coffe and Cheesecake

Asignatura: Laboratorio Web 1 — Tecnicatura Universitaria en Desarrollo Web.

Profesor: Jorge Ceferino Valdez.

Público objetivo: Alumnos que recién toman contacto con HTML5, CSS3 vanilla y JavaScript vanilla que desean consolidar sus conocimientos mediante un proyecto práctico y progresivo.

Tecnologías: HTML5, CSS3 vanilla, JavaScript vanilla (sin frameworks ni librerías).

Enfoque pedagógico: Cada tarea se apoya en la anterior, permitiendo ver la evolución de un “sitio estático” a un “sitio web interactivo y responsive”. No se introducen patrones de diseño formales; sí se muestran patrones de maquetación y se explica cada decisión de código de manera didáctica.

Tabla de contenidos

1. Visión general del proyecto
2. Definición del Proyecto
3. Cómo leer esta guía
4. Diagrama de Maquetación
5. Fase 1 — Maquetación Estructural (HTML)
6. Fase 2 — Aplicación de Estilos (CSS)
7. Fase 3 — Responsive y Menú Móvil
8. Fase 4 — Interactividad con JavaScript
 - Técnica: Lightbox
 - Técnica: Slider de imágenes
 - Técnica: Tabs
 - Técnica: Parallax Scroll
 - Técnica: Material Design en formularios
 - Técnica: Validación de formulario
 - Técnica: Scroll suave
 - Técnica: Menú móvil deslizable
9. Arquitectura final: decisiones clave
10. Checklist de aprendizaje

1. Visión general del proyecto

Este ejercicio construye un sitio web de una cafetería ficticia llamada “**Coffe and Cheesecake**”. Al finalizar todas las fases, el sitio permite:

- Visualizar una landing page con 7 secciones: Encabezado, Portada, Nosotros, Nuestro Chef, Menú, Contacto y Pie de página.
- Explorar imágenes de la portada con un efecto **Lightbox** (ventana modal ampliada).
- Ver un **slider** automático de imágenes del chef.
- Navegar el menú mediante **Tabs** (Postres y Cafés).
- Experimentar un efecto **Parallax** en la sección de contacto.
- Usar un formulario con **Material Design** (etiquetas flotantes).
- Validar el formulario con **JavaScript** antes de enviarlo.
- Desplazarse por la página con **scroll suave** desde los enlaces del menú.
- Usar un **menú móvil** con deslizamiento lateral (hamburguesa).
- Adaptarse automáticamente a diferentes tamaños de pantalla mediante **Media Queries**.

El proyecto se compone de tres archivos de código, uno por cada **capa** de una página web:

Archivo	Capa	Responsabilidad
index.html	Estructura	Qué elementos existen, su semántica y su jerarquía
css/estilos.css	Presentación	Cómo se ven esos elementos, layout responsive y efectos visuales
js/*.js	Conducta y datos	Qué hace la página: interacciones, animaciones, validación

La idea central de la cursada es que cada paso **agrega una capacidad** sin romper lo anterior. El sitio es visible desde la primera clase; lo que cambia es cuánto puede hacer y cuán bien se ve.

2. Definición del Proyecto

“**Coffe and Cheesecake**” es un sitio web de una página (landing page) para una cafetería ficticia que combina la elegancia de la pastelería artesanal con la calidez de un buen café. El proyecto nace como ejercicio académico para practicar maquetación semántica, diseño responsive e interactividad con JavaScript vanilla.

Decisiones de diseño

El diseño prioriza la **elegancia visual** sobre la complejidad. Se optó por un estilo minimalista y sofisticado que evoca las cafeterías boutique europeas:

- **Tipografía dual:** Playfair Display (serif) para títulos y encabezados, transmitiendo tradición y distinción; Poppins (sans-serif) para cuerpos de texto, menú y formularios, aportando legibilidad moderna.
- **Espaciado generoso:** Amplios márgenes verticales (padding: 5.625rem) entre secciones para que el contenido “respire” y no se sature visualmente.
- **Contraste claro-oscuro:** Alternancia intencional entre fondos blancos (Nosotros, Menú) y fondos oscuros (Portada, Chef) para crear ritmo visual y guiar la atención del usuario.
- **Imágenes como fondo:** Las fotografías de la portada y del chef se integran como background-image, permitiendo que los overlays y textos se superpongan de forma elegante sin romper el flujo del documento.
- **Navegación persistente:** El menú fijo (position: fixed) garantiza que el usuario siempre tenga acceso a las secciones principales, sin importar dónde se encuentre en la página.

Paleta de colores

Variable CSS	Valor	Uso
--color-negro-custom	#141618	Fondos oscuros (Portada, Chef, Footer)

Variable CSS	Valor	Uso
--color-blanco	#fff	Fondos claros y texto sobre oscuro
--color-texto-oscuro	#303133	Textos principales sobre fondo claro
--color-texto-claro	#777	Textos secundarios, descripciones, labels
--color-dorado	#b4975a	Acentos: botón de envío, borde activo de tabs, hover en redes
--color-gris-claro	#eaeaea	Bordes, separadores, contornos de blurbs

La elección del **dorado** (#b4975a) como color de acento no es casual: evoca la calidez del café recién servido y el brillo sutil de una cafetería premium. El negro casi puro (#141618) evita la dureza del negro absoluto (#000) y aporta sofisticación. Los grises intermedios (#777, #eaeaea) equilibran el contraste sin competir visualmente con los elementos principales.

Estilos visuales e identidad

La identidad del sitio se construye sobre tres pilares:

1. **Clasicismo tipográfico:** Los títulos en itálica (`font-style: italic`) con `letter-spacing: -0.02em` evocan las portadas de revistas gastronómicas.
2. **Interactividad sutil:** Los overlays de la portada aparecen con `opacity` y `transition`, invitando al clic sin agredir la vista. El lightbox, el slider y el parallax añaden dinamismo controlado.
3. **Responsive como regla, no como excepción:** Desde el primer paso se pensó en dos versiones del menú (PC y móvil) y en cómo las columnas se apilan verticalmente en pantallas pequeñas. Las Media Queries no son un “arreglo” al final: son parte integral del diseño.

El resultado es un sitio que se siente **premium pero accesible**: sofisticado en sus detalles, pero claro en su estructura para que un alumno principiante pueda entender cada decisión y replicarla.

3. Cómo leer esta guía

Cada fase sigue siempre el mismo esquema:

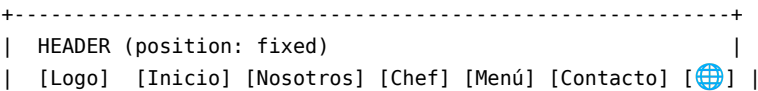
1. **Objetivo** — qué problema concreto vamos a resolver.
2. **Pasos** — el código, presentado en el orden en que conviene escribirlo.
3. **Decisiones de implementación** — el porqué de **cada línea**. Esta es la parte más importante: copiar código es fácil; entender por qué está escrito así es lo que se evalúa.

Una recomendación de método: **escribí el código vos mismo**, no lo copies y pegues. Al teclear cada línea te vas a hacer preguntas (“¿por qué `position: fixed` en el header?”, “¿por qué `box-sizing: border-box`?”) y esta guía las responde justo al lado.

Vas a ver un patrón recurrente en JavaScript: **separar propiedades (datos) de métodos (funciones)** en dos objetos. Esto no es un framework; es una convención didáctica para que puedas identificar qué cambia (propiedades) y qué hace algo (métodos).

4. Diagrama de Maquetación

A continuación se presenta un diagrama visual simplificado que muestra cómo se distribuyen los elementos estructurales del sitio “Coffe and Cheesecake”.



```

+-----+
|
| PORTADA
| +-----+ +-----+
| | Foto 1      | | Foto Full 1  |
| | (lightbox)  | | (lightbox)  |
| +-----+ +-----+
| | Texto      | | Foto 2      |
| | "Mejor Calidad" | | (lightbox)  |
| +-----+ +-----+
| | Foto Full 2 | | Texto      |
| | (lightbox)  | | "Mejor Calidad" |
| +-----+ +-----+
|
+-----+

|
| NOSOTROS
| +-----+ +-----+
| | Nosotros | |
| | Conócenos | | Texto descriptivo de la cafetería |
| +-----+ +-----+
|
+-----+

|
| NUESTRO CHEF (fondo oscuro)
| +-----+ +-----+
| |
| | SLIDER de imágenes | | Nuestro Chef
| | [Chef 1] [Chef 2] ... | | Texto descriptivo
| |
| +-----+ +-----+
|
+-----+

|
| NUESTRO MENÚ
| [ Postres | Cafés ]
| +-----+
| | Cheesecake ..... $5.00
| | Tiramisú ..... $5.00
| | ...
| +-----+
|
+-----+

|
| CONTACTO
| +-----+ +-----+ +-----+
| |Escríbenos| |Visítanos| |Horario | <- (Parallax) |
| +-----+ +-----+ +-----+
|
| +-----+
| | Nombre
| | Correo electrónico
| | Mensaje
| | [ Enviar ]
| +-----+
|

```

+-----+		
+-----+		
	FOOTER	
	Todos los derechos reservados [fb] [tw] [ig]	
+-----+		

Elementos clave del diagrama:

Sección	Técnica asociada	Rol en el sitio
Header	position: fixed, Flexbox	Navegación persistente y accesible
Portada	Lightbox, Grid de 2 columnas	Primera impresión visual; galería interactiva
Nosotros	Flexbox, borde separador	Presentación de la marca y valores
Chef	Slider automático	Mostrar al equipo humano detrás del producto
Menú	Tabs (pestañas)	Organizar contenido largo en categorías
Contacto	Parallax, Material Design, Validación	Información de contacto + interacción directa
Footer	Iconos sociales, hover effects	Cierre institucional y enlaces externos

5. Fase 1 — Maquetación Estructural (HTML)

Objetivo

Crear la estructura semántica del documento HTML, incluyendo el encabezado con navegación, las secciones de contenido y el pie de página. En esta fase no aplicamos estilos visuales: solo organizamos los elementos para que el navegador los interprete correctamente.

Paso 1.1: Estructura base del documento

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0, user-scalable=no, minimum-scale=1.0, maximum-scale=1.0">
  <meta name="author" content="UNPA - UACO">
  <meta name="keywords" content="desarrollo web, laboratorio web I, UNPA, UACO">
  <meta name="description" content="Es una asignatura de la tecnicatura universitaria en desarrollo web.">
  <link rel="stylesheet" href="css/font-awesome.min.css">
  <link href="https://fonts.googleapis.com/css?family=Playfair+Display:400,700|Poppins" rel="stylesheet">
  <link rel="stylesheet" href="css/estilos.css">
  <title>Coffe and Cake</title>
</head>
<body>

</body>
</html>
```

Decisiones de implementación, línea por línea:

- `<meta charset="utf-8">`: permite mostrar correctamente caracteres especiales del español (tildes, eñes).
- `<meta name="viewport" content="width=device-width, initial-scale=1.0...">`: instrucción clave para que el sitio sea **responsive**. Le dice al navegador móvil que el ancho del sitio debe coincidir con el ancho del dispositivo.
- `<link rel="stylesheet" href="css/font-awesome.min.css">`: librería de iconos (fuentes) que usaremos para los íconos de redes sociales y el botón de cierre del Lightbox. Es la única dependencia externa del proyecto.
- Google Fonts (Playfair Display y Poppins): tipografías que le dan identidad visual al sitio. Playfair es una serif elegante; Poppins es una sans-serif moderna y legible.

- `<link rel="stylesheet" href="css/estilos.css">`: nuestro archivo principal de estilos. Siempre va **después** de las librerías externas para que nuestras reglas tengan prioridad.

Paso 1.2: Encabezado (`<header>`)

El encabezado contiene el logo, el menú de navegación principal y los iconos de redes sociales. Es el primer elemento visible y se mantendrá **fijo** en la parte superior de la pantalla.

```
<header>
  <div class="menu-bar-pc">
    <a href="#" class="logo">
      
    </a>

    <nav class="menu-principal">
      <a href="#">Inicio</a>
      <a href="#">Nosotros</a>
      <a href="#">Nuestro Chef</a>
      <a href="#">Menú</a>
      <a href="#">Contacto</a>
    </nav>

    <div class="top-redes">
      <a href="#"><i class="fa fa-facebook" aria-hidden="true"></i></a>
      <a href="#"><i class="fa fa-twitter" aria-hidden="true"></i></a>
      <a href="#"><i class="fa fa-instagram" aria-hidden="true"></i></a>
    </div>
  </div>
</header>
```

Decisiones de implementación:

- Usamos `<nav>` para el menú principal porque es la etiqueta semántica correcta para bloques de navegación.
- Los iconos de Font Awesome se insertan mediante la etiqueta `<i>` con clases específicas (`fa fa-facebook`, etc.). El atributo `aria-hidden="true"` indica que son decorativos y no deben ser leídos por lectores de pantalla.
- El `alt=""` vacío en la imagen del logo es una decisión temporal; idealmente debería describir el contenido de la imagen.

Paso 1.3: Sección Portada (`<section class="portada">`)

La portada es una galería visual compuesta por dos columnas (`col`) que alternan fotos y textos. Las fotos usarán **imágenes de fondo** (CSS `background-image`) en lugar de etiquetas `<img>`, lo que permite que el texto se superponga de forma elegante.

```
<section class="main">
  <section class="portada" id="portada">
    <div class="col">
      <div class="foto izq lightbox">
        <div class="overlay">
          <i class="fa fa-plus" aria-hidden="true"></i>
        </div>
      </div>
      <div class="texto">
        <h2>Mejor Calidad</h2>
        <p>Lorem ipsum dolor, sit amet consectetur adipisicing elit...</p>
      </div>
      <div class="foto-full izq lightbox">
        <div class="overlay">
          <i class="fa fa-plus" aria-hidden="true"></i>
        </div>
      </div>
    </div>
  </section>
</section>
```

```

    <div class="col">
      <!-- Fotos y texto simétricos -->
    </div>
  </section>
</section>

```

Decisiones de implementación:

- Cada `.foto` y `.foto-full` tiene la clase `.lightbox` porque más adelante les aplicaremos el efecto de ampliación al hacer clic.
- El `.overlay` es una capa semitransparente que aparece al pasar el mouse (efecto hover) e indica que la imagen es interactiva. El icono `fa-plus` sugiere que se puede “ampliar”.
- El `id="portada"` servirá más adelante para el **scroll suave**: los enlaces del menú apuntarán a este identificador.

Paso 1.4: Secciones Nosotros, Chef, Menú y Contacto

Estas secciones siguen la misma lógica semántica: un `<section>` con un identificador único, un contenedor `.container` que centra el contenido, y columnas `.col` que distribuyen la información.

Nosotros:

```

<section class="nosotros" id="nosotros">
  <div class="container">
    <div class="col izq">
      <div class="titulo-seccion">
        <p>Nosotros</p>
        <h2>Conócenos</h2>
      </div>
    </div>
    <div class="col der">
      <div class="texto">
        <p>Lorem ipsum dolor sit, amet consectetur adipisicing elit...</p>
      </div>
    </div>
  </div>
</section>

```

Nuestro Chef (con slider):

```

<section class="chef" id="chef">
  <div class="container">
    <div class="col izq">
      <div class="slider" id="slider">
        <div class="slide foto1"></div>
        <div class="slide foto2"></div>
        <div class="slide foto3"></div>
      </div>
    </div>
    <div class="col">
      <div class="titulo-seccion">
        <h2>Nuestro Chef</h2>
        <p>...</p>
      </div>
    </div>
  </div>
</section>

```

Decisiones de implementación:

- El slider está compuesto por tres `div` con clase `.slide` dentro de un contenedor `#slider`. No usamos etiquetas `<img>`; las imágenes se cargarán como fondos CSS. Esto facilita la transición entre slides.
- El `id="slider"` es obligatorio porque JavaScript lo utilizará para calcular el ancho y desplazar las diapositivas.

Nuestro Menú (con Tabs):

```
<section id="nuestro_menu" class="menu-platos">
  <div class="container">
    <div class="titulo-seccion"><h2>Nuestro menú</h2></div>
    <div class="contenedor-menu">
      <ul id="encabezado_menu" class="encabezado">
        <li class="active"><a href="#postres">Postres</a></li>
        <li><a href="#cafes">Cafés</a></li>
      </ul>
      <div class="contenido" id="contenido_menu">
        <div id="postres" class="active">
          <!-- Items de postres -->
        </div>
        <div id="cafes">
          <!-- Items de cafés -->
        </div>
      </div>
    </div>
  </div>
</section>
```

Decisiones de implementación:

- El sistema de Tabs se basa en dos partes: un **encabezado** (#encabezado_menu) con enlaces que apuntan a href="#postres" y href="#cafes", y un **contenedor** (#contenido_menu) con dos div cuyos id coinciden con esos href.
- La clase .active indica cuál pestaña y cuál contenido están visibles inicialmente.
- Cada .item del menú está dividido en dos columnas: .col.izq (nombre y descripción) y .col.der (precio).

Contacto (con parallax y formulario):

```
<section class="contacto" id="contacto">
  <div class="datos parallax">
    <div class="overlay"></div>
    <div class="container">
      <div class="blurb"><h3>Escríbenos</h3><p>correo@correo.com</p></div>
      <div class="blurb"><h3>Visítanos</h3><p>Calle 123 #27-27<br>Ciudad X</p></div>
      <div class="blurb"><h3>Horario</h3><p>Martes a Domingo<br>11am - 9pm</p></div>
    </div>
  </div>
  <div class="formulario">
    <form action="" class="formulario_contacto" name="formulario_contacto">
      <div class="input-group">
        <input type="text" id="nombre" name="nombre">
        <label class="label" for="nombre">Nombre</label>
      </div>
      <div class="input-group">
        <input type="email" id="email" name="email">
        <label class="label" for="email">Correo electrónico</label>
      </div>
      <div class="input-group">
        <textarea name="mensaje" id="mensaje"></textarea>
        <label class="label" for="mensaje">Mensaje</label>
      </div>
      <input type="submit" value="Enviar">
    </form>
  </div>
</section>
```

Decisiones de implementación:

- `.datos.parallax`: la clase `parallax` servirá para aplicar el efecto de desplazamiento del fondo más lento que el scroll.
- `.overlay`: es una capa oscura semitransparente que mejora la legibilidad del texto sobre la imagen de fondo.
- El `name="formulario_contacto"` es crucial: JavaScript accederá al formulario mediante `document.formulario_contacto`.
- Cada `.input-group` contiene un `<input>` (o `<textarea>`) seguido de su `<label>`. El label está **después** del input en el HTML, pero se posicionará **encima** mediante CSS (efecto Material Design flotante).

Paso 1.5: Pie de página (<footer>)

```
<footer>
  <p class="copyright">Todos los derechos reservados</p>
  <div class="bottom-redes">
    <a href="#"><i class="fa fa-facebook" aria-hidden="true"></i></a>
    <a href="#"><i class="fa fa-twitter" aria-hidden="true"></i></a>
    <a href="#"><i class="fa fa-instagram" aria-hidden="true"></i></a>
  </div>
</footer>
```

6. Fase 2 — Aplicación de Estilos (CSS)

Objetivo

Transformar la estructura HTML plana en un sitio visualmente atractivo, aplicando colores, tipografías, espaciados, flexbox para el layout y preparando las bases para los efectos interactivos.

Paso 2.1: Variables CSS (Custom Properties)

```
:root {
  --color-negro-custom: #141618;
  --color-blanco: #fff;
  --color-texto-oscuro: #303133;
  --color-texto-claro: #777;
  --color-dorado: #b4975a;
  --color-gris-claro: #eaeaea;
  --fuente-principal: 'Playfair Display', serif;
  --fuente-secundaria: 'Poppins', serif;
}
```

Decisiones de implementación:

- Las **variables CSS** se definen dentro de `:root` para que estén disponibles en todo el documento.
- Este patrón permite cambiar un color una sola vez y ver el efecto en todas las reglas que lo usan. Por ejemplo, si queremos cambiar el dorado por otro tono, solo editamos `--color-dorado`.
- La paleta busca transmitir **elegancia y calidez**: negro profundo para contrastes, dorado como color de acento, grises para textos secundarios.

Paso 2.2: Estilos generales y reseteo

```
* {
  margin: 0;
  padding: 0;
  -webkit-box-sizing: border-box;
  -moz-box-sizing: border-box;
  box-sizing: border-box;
}
body {
  font-size: 16px;
```

```

font-family: var(--fuente-principal);
color: var(--color-texto-oscuro);
}
.container {
width: 90%;
max-width: 1080px;
margin: 0 auto;
}

```

Decisiones de implementación:

- box-sizing: border-box es esencial: hace que el padding y el border se incluyan dentro del ancho total del elemento, evitando cálculos matemáticos complicados.
- .container es un patrón recurrente: limita el ancho máximo a 1080px y centra el contenido horizontalmente con margin: 0 auto.

Paso 2.3: Encabezado fijo (Header)

```

header .menu-bar-pc {
background: var(--color-blanco);
width: 100%;
position: fixed;
z-index: 99;
top: 0;
padding: 0 2.25rem;
display: flex;
flex-direction: row;
justify-content: space-between;
align-items: center;
min-height: 5.875rem;
border-bottom: 1px solid var(--color-gris-claro);
}

```

Decisiones de implementación:

- position: fixed saca al encabezado del flujo normal del documento y lo “pega” a la ventana del navegador, siempre visible al hacer scroll.
- z-index: 99 asegura que el header quede por encima de otros elementos (como el lightbox, que usará z-index: 1000 cuando aparezca).
- **Flexbox** (display: flex) distribuye los tres elementos (logo, menú, redes) en una fila, con justify-content: space-between para que ocupen todo el ancho y queden separados.

Paso 2.4: Sección Portada

```

.main {
margin-top: 5.875rem; /* Deja espacio para el header fijo */
}
.main .portada {
display: flex;
flex-wrap: wrap;
justify-content: space-between;
}
.main .portada .col {
width: 50%;
background: var(--color-negro-custom);
color: var(--color-blanco);
display: flex;
flex-wrap: wrap;
}

```

```

.main .portada .col .foto {
  background-size: cover;
  background-position: center;
  width: 50%;
}
.main .portada .col .foto.izq {
  background-image: url('../img/portada-1.jpg');
}
.main .portada .col .foto.der {
  background-image: url('../img/portada-4.jpg');
}
.main .portada .col .foto-full {
  width: 100%;
  min-height: 15.25rem;
  background-size: cover;
}

```

Decisiones de implementación:

- El margin-top: 5.875rem en .main es crucial: como el header es fijo y mide aproximadamente 94px (5.875rem), si no agregamos este margen, la portada quedaría tapada por el menú.
- Cada .col mide el 50 % del ancho, creando una **grilla de dos columnas**.
- Dentro de cada columna, las fotos miden 50 % (foto normal) o 100 % (foto-full). Esto genera un patrón visual tipo “ajedrez”.

Paso 2.5: Overlay interactivo (preparando el Lightbox)

```

.main .portada .col .foto .overlay {
  opacity: 0;
  width: 100%;
  height: 100%;
  background: rgba(20, 22, 24, 0.4);
  position: relative;
  cursor: pointer;
  transition: all 0.3s ease;
}
.main .portada .col .foto .overlay:hover {
  opacity: 1;
}
.main .portada .col .foto .overlay i {
  position: absolute;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
  font-size: 1.5625rem;
}

```

Decisiones de implementación:

- El .overlay comienza con opacity: 0 (invisible). Al pasar el mouse (:hover), cambia a opacity: 1 (visible). La transición suaviza este cambio en 0.3 segundos.
- El icono fa-plus se centra perfectamente con la combinación de position: absolute + transform: translate(-50%, -50%). Esta es una técnica estándar para centrar elementos respecto a su padre.

Paso 2.6: Sección Nosotros

```

.main .nosotros {
  padding: 5.625rem 0;
}
.main .nosotros .container {

```

```

display: flex;
flex-direction: row;
flex-wrap: wrap;
}
.main .nosotros .container .col.izq {
width: 30%;
border-right: 1px solid var(--color-gris-claro);
display: flex;
align-items: center;
padding-right: 5%;
}
.main .nosotros .container .col.der {
width: 70%;
padding-left: 5%;
}

```

Decisiones de implementación:

- La columna izquierda ocupa el 30 % y tiene un borde derecho sutil que actúa como separador visual.
- La columna derecha ocupa el 70 % y contiene el texto descriptivo. La suma (30 % + 70 %) da 100 %, pero el padding y el border no rompen el layout gracias a box-sizing: border-box.

Paso 2.7: Sección Chef y Slider

```

.main .chef {
background: var(--color-negro-custom);
padding: 5.625rem 0;
}
.main .chef .container .col.izq {
box-shadow: 0px 9px 32px -1px rgba(0, 0, 0, 0.5);
overflow: hidden;
}
.main .chef .container .col .slider {
width: 300%;
display: flex;
}
.main .chef .container .col .slider .slide {
width: 100%;
min-height: 400px;
background-size: cover;
background-position: center;
}
.main .chef .container .col .slider .slide.foto1 {
background-image: url('../img/chef-1.jpg');
}

```

Decisiones de implementación:

- El slider funciona como una **tira de imágenes** horizontal. Si hay 3 slides, el contenedor mide 300 % del ancho de una sola columna. Cada slide mide 100 % de esa columna.
- JavaScript desplazará esta tira moviendo el margin-left del contenedor #slider.
- overflow: hidden en la columna izquierda asegura que las slides que “sobran” a la derecha no se vean fuera del recuadro.

Paso 2.8: Sección Menú (preparando los Tabs)

```

.main .menu-platos .container .contenedor-menu .encabezado {
list-style: none;
display: flex;
justify-content: center;
}

```

```

}
.main .menu-platos .container .contenedor-menu .encabezado li a {
  font-size: 0.75rem;
  padding: 0 1.4375rem 1.125rem 1.4375rem;
  display: block;
  cursor: pointer;
  border-bottom: 2px solid transparent;
}
.main .menu-platos .container .contenedor-menu .encabezado li.active a {
  color: var(--color-dorado);
  border-bottom: 2px solid var(--color-dorado);
}
.main .menu-platos .container .contenedor-menu .contenido > div {
  display: none;
}
.main .menu-platos .container .contenedor-menu .contenido .active {
  display: block;
}

```

Decisiones de implementación:

- Por defecto, todos los paneles de contenido (> div) tienen display: none, por lo que están ocultos.
- Solo el panel que tenga la clase .active se mostrará (display: block).
- JavaScript se encargará de quitar y agregar la clase .active a los li del encabezado y a los div del contenido.

Paso 2.9: Sección Contacto y Formulario

```

.main .contacto .datos {
  padding: 5.625rem 0 11.875rem 0;
  position: relative;
  background-image: url('../img/contacto-bg.jpg');
  background-size: cover;
  background-position: center;
}
.main .contacto .datos .overlay {
  position: absolute;
  width: 100%;
  height: 100%;
  background: rgba(20, 22, 24, 0.3);
  top: 0;
  z-index: 1;
}
.main .contacto .datos .container {
  position: relative;
  z-index: 2;
}
.main .contacto .formulario {
  position: relative;
  z-index: 3;
  background: var(--color-blanco);
  max-width: 40.625rem;
  margin: 0 auto;
  margin-top: -6.25rem;
  width: 90%;
}

```

Decisiones de implementación:

- El formulario usa margin-top: -6.25rem para **subirse** y superponerse parcialmente sobre la sección de datos. Este efecto de “tarjeta flotante” es común en diseños modernos.

- El `.overlay` tiene `position: absolute` y cubre toda la sección. Los textos (`.blurb`) tienen `z-index: 2` para quedar por encima.

Paso 2.10: Pie de página

```
footer {
  margin-top: 2.25rem;
  background: var(--color-negro-custom);
  padding: 5.625rem 0;
  text-align: center;
  color: var(--color-blanco);
}
footer .bottom-redes a {
  padding: 0 0.5625rem;
  color: var(--color-blanco);
  transition: all 0.3s ease;
}
footer .bottom-redes a:hover {
  color: var(--color-dorado);
}
```

7. Fase 3 — Responsive y Menú Móvil

Objetivo

Hacer que el sitio se vea bien en cualquier dispositivo, desde monitores grandes hasta celulares, utilizando **Media Queries** y un menú de navegación alternativo para móviles.

Técnica: Media Queries

Las **Media Queries** son reglas CSS que solo se aplican cuando se cumple una condición (por ejemplo, “si el ancho de pantalla es menor a 800px”). Nos permiten cambiar estilos sin alterar el HTML.

```
@media all and (max-width: 800px) {
  header .menu-bar-pc {
    display: none;
  }
  header .menu-bar-movil {
    display: flex;
  }
  .main .portada .col {
    width: 100%;
  }
  .main .chef .container .col {
    width: 100%;
  }
}
```

Decisiones de implementación:

- `@media all and (max-width: 800px)`: esta regla agrupa todos los ajustes para pantallas pequeñas. El punto de corte (800px) se eligió porque es donde las dos columnas de la portada empiezan a verse comprimidas.
- En móvil, `.menu-bar-pc` (PC) se oculta (`display: none`) y aparece `.menu-bar-movil` (`display: flex`).
- Las columnas de la portada y del chef pasan de 50 % a 100 % de ancho, apilándose verticalmente.

Técnica: Menú móvil deslizable

El menú móvil consiste en una barra superior con un ícono de hamburguesa (`fa-bars`) y un panel lateral (`.slideMenu`) que inicialmente está fuera de la pantalla.

HTML del menú móvil:

```
<div class="menu-bar-movil">
  <a href="#" class="logo"></a>
  <div class="burger-menu" id="burger_menu">
    <i class="fa fa-bars" aria-hidden="true"></i>
  </div>
  <div class="slideMenu" id="slideMenu">
    <div class="top-redes">...</div>
    <nav class="menu-principal">...</nav>
  </div>
</div>
```

CSS del menú móvil:

```
header .menu-bar-movil {
  display: none; /* Oculto en PC */
  /* ... mismas propiedades que menu-bar-pc ... */
}
header .menu-bar-movil .slideMenu {
  position: fixed;
  top: 5.875rem;
  left: -80%;
  background: var(--color-blanco);
  width: 80%;
  transition: all 0.3s ease;
}
header .menu-bar-movil .slideMenu.active {
  left: 0;
}
```

Decisiones de implementación:

- .slideMenu comienza con left: -80%, es decir, fuera de la pantalla hacia la izquierda.
- Cuando se le agrega la clase .active, pasa a left: 0, deslizándose visualmente desde la izquierda.
- La transición de 0.3 segundos suaviza el movimiento.

8. Fase 4 — Interactividad con JavaScript

Técnica: Lightbox

¿Qué es? Un **Lightbox** es una ventana modal que muestra una imagen ampliada al hacer clic en una versión pequeña (thumbnail). Oscurece el fondo y permite cerrarla con un botón.

¿Por qué la aplicamos? En la portada tenemos varias fotos que actúan como “avance” del local. El lightbox permite al usuario verlas en detalle sin salir de la página.

Código JavaScript

```
var proplightbox = {
  imgContainer: document.getElementsByClassName('lightbox'),
  imagen: null,
  imagenSrc: null,
  cuerpoDom: document.getElementsByTagName('body')[0],
  lightbox_container: null,
  modal: null,
  cerrarModal: null,
  animacion: 'fade'
};
```

```

var metLightbox = {
  inicio: function(){
    for (var i = 0; i < propLightbox.imgContainer.length; i++) {
      propLightbox.imgContainer[i].addEventListener('click', metLightbox.capturaImagen);
    }
  },

  capturaImagen: function(){
    propLightbox.imagen = this;
    metLightbox.lightbox(propLightbox.imagen);
  },

  lightbox: function(imagen){
    // Extraemos la URL del background-image con slice(5, -2)
    propLightbox.imagenSrc = window.getComputedStyle(imagen, null).backgroundImage.slice(5, -2);

    // Creamos el contenedor oscuro
    propLightbox.cuerpoDom.appendChild(document.createElement('DIV')).setAttribute('id', 'lightbox_container');
    propLightbox.lightbox_container = document.getElementById('lightbox_container');
    propLightbox.lightbox_container.style.width = '100%';
    propLightbox.lightbox_container.style.height = '100%';
    propLightbox.lightbox_container.style.position = 'fixed';
    propLightbox.lightbox_container.style.zIndex = '1000';
    propLightbox.lightbox_container.style.background = 'rgba(0,0,0,0.8)';
    propLightbox.lightbox_container.style.top = '0';
    propLightbox.lightbox_container.style.left = '0';

    // Creamos la ventana modal
    propLightbox.lightbox_container.appendChild(document.createElement('DIV')).setAttribute('id', 'modal');
    propLightbox.modal = document.getElementById('modal');
    propLightbox.modal.style.height = '100%';

    // Insertamos la imagen
    propLightbox.modal.appendChild(document.createElement('IMG')).setAttribute('src', propLightbox.imagenSrc);
    propLightbox.modal.getElementsByTagName('img')[0].setAttribute('class', 'imagen-modal');

    // Animación fade
    if (propLightbox.animacion == 'fade'){
      document.getElementsByClassName('imagen-modal')[0].style.opacity = 0;
      setTimeout(function () {
        document.getElementsByClassName('imagen-modal')[0].style.opacity = 1;
      }, 50);
    }

    // Botón de cierre
    propLightbox.modal.innerHTML += '<i id="cerrar_modal" class="fa fa-times" aria-hidden="true"></i>';
    propLightbox.cerrarModal = document.getElementById('cerrar_modal');
    propLightbox.cerrarModal.addEventListener('click', metLightbox.cerrarModal);
  },

  cerrarModal: function(){
    propLightbox.cuerpoDom.removeChild(propLightbox.lightbox_container);
  }
};

metLightbox.inicio();

```

Decisiones de implementación:

- `window.getComputedStyle(imagen, null).backgroundImage` obtiene la URL de la imagen de fondo aplicada por CSS. Como

devuelve url("..."), usamos `.slice(5, -2)` para quitar los primeros 5 caracteres (`url(")`) y los últimos 2 (`")`)).

- Creamos los elementos (DIV, IMG) con `document.createElement` en lugar de escribir HTML plano para tener control total sobre sus atributos.
 - La animación fade usa `opacity`: comienza en 0 (invisible) y pasa a 1 (visible) después de 50 milisegundos, produciendo un suave efecto de aparición.
 - Al cerrar, simplemente removemos el contenedor del DOM con `removeChild`, liberando memoria.
-

Técnica: Slider de imágenes

¿Qué es? Un **slider** (o carrusel) es un componente que muestra una serie de imágenes una tras otra, desplazándolas automáticamente.

¿Por qué la aplicamos? En la sección “Nuestro Chef” queremos mostrar varias fotos del chef sin ocupar mucho espacio vertical.

Código JavaScript

```
var propSlider = {
  slider: document.getElementById('slider'),
  primerSlide: null
};

var metSlider = {
  inicio: function () {
    setInterval(metSlider.moverSlide, 3000);
  },

  moverSlide: function () {
    propSlider.slider.style.transition = 'all 1s ease';
    propSlider.slider.style.marginLeft = '-100%';

    setTimeout(function () {
      propSlider.primerSlide = propSlider.slider.firstElementChild;
      propSlider.slider.appendChild(propSlider.primerSlide);

      propSlider.slider.style.transition = 'unset';
      propSlider.slider.style.marginLeft = 0;
    }, 1000);
  }
};

metSlider.inicio();
```

Decisiones de implementación:

- `setInterval(metSlider.moverSlide, 3000)` ejecuta la función cada 3 segundos, creando el ciclo infinito del slider.
 - El truco del slider es mover el contenedor `margin-left: -100%`, lo que desplaza la primera imagen fuera de la vista.
 - Después de 1 segundo (tiempo que dura la transición), **movemos la primera slide al final** del contenedor (`appendChild(primerSlide)`) y reseteamos el `margin-left` a 0. Esto crea una rotación continua sin fin.
 - `transition: 'unset'` desactiva momentáneamente la animación para que el reseteo del `margin-left` no sea visible.
-

Técnica: Tabs

¿Qué es? Los **Tabs** (pestañas) permiten organizar contenido en paneles intercambiables. Solo un panel es visible a la vez.

¿Por qué la aplicamos? El menú tiene dos categorías: Postres y Cafés. En lugar de mostrar todo junto (muy largo), agrupamos cada categoría en su propia pestaña.

Código JavaScript

```
(function(){
    var propTabs = {
        primer_encabezado: document.getElementById('encabezado_menu').firstElementChild,
        primer_contenido: document.getElementById('contenido_menu').firstElementChild,
        enlaces_encabezado: document.querySelectorAll('#encabezado_menu li a'),
        li_encabezado: document.querySelectorAll('#encabezado_menu li'),
        divs_contenido: document.querySelectorAll('#contenido_menu > div'),
        contenido_activo: null
    };

    var metTabs = {
        inicio: function(){
            propTabs.primer_encabezado.className = 'active';
            propTabs.primer_contenido.className = 'active';

            for (var i = 0; i < propTabs.enlaces_encabezado.length; i++) {
                propTabs.enlaces_encabezado[i].addEventListener('click', metTabs.evento);
            }
        },

        evento: function(e){
            e.preventDefault();

            // Limpiamos todas las clases active
            for (var i = 0; i < propTabs.li_encabezado.length; i++) {
                propTabs.li_encabezado[i].className = '';
            }
            for (var i = 0; i < propTabs.divs_contenido.length; i++) {
                propTabs.divs_contenido[i].className = '';
            }

            // Activamos el elemento clickeado
            this.parentElement.className = 'active';
            propTabs.contenido_activo = this.getAttribute('href');
            document.querySelector(propTabs.contenido_activo).className = 'active';

            // Animación de fade
            document.querySelector(propTabs.contenido_activo).style.opacity = 0;
            setTimeout(function(){
                document.querySelector(propTabs.contenido_activo).style.opacity = 1;
            }, 100);
        }
    };

    metTabs.inicio();
})();
```

Decisiones de implementación:

- Usamos una **IIFE** (Immediately Invoked Function Expression, `(function(){...})()`) para encapsular todo el código. Esto evita que las variables `propTabs` y `metTabs` polluen el ámbito global.
- `e.preventDefault()` evita que el navegador siga el enlace (`href="#postres"`) y haga scroll brusco. Nosotros controlamos manualmente qué contenido mostrar.
- `this.getAttribute('href')` obtiene el valor del atributo `href` del enlace clickeado (por ejemplo, `"#postres"`). Luego usamos `document.querySelector("#postres")` para encontrar el panel correspondiente.
- La animación de fade al cambiar de pestaña mejora la experiencia visual.

Técnica: Parallax Scroll

¿Qué es? El efecto **Parallax** hace que el fondo de una sección se mueva más lentamente que el contenido al hacer scroll, creando una sensación de profundidad.

¿Por qué la aplicamos? La sección de contacto tiene una imagen de fondo. El parallax la convierte en un elemento dinámico y elegante.

Código JavaScript

```
(function(){
    var propParallax = {
        seccion: document.querySelector('.parallax'),
        recorrido: null,
        limite: null
    };

    var metParallax = {
        inicio: function () {
            window.addEventListener('scroll', metParallax.scrollParallax);
        },

        scrollParallax: function () {
            propParallax.recorrido = window.pageYOffset;
            propParallax.limite = propParallax.seccion.offsetTop + propParallax.seccion.offsetHeight;

            if (propParallax.recorrido > propParallax.seccion.offsetTop - window.outerHeight
                && propParallax.recorrido <= propParallax.limite) {

                propParallax.seccion.style.backgroundPositionY =
                    (propParallax.recorrido - propParallax.seccion.offsetTop) / 1.5 + 'px';
            } else {
                propParallax.seccion.style.backgroundPositionY = 0;
            }
        }
    };

    metParallax.inicio();
})();
```

Decisiones de implementación:

- `window.pageYOffset` nos dice cuántos píxeles ha bajado el usuario desde la parte superior de la página.
- Solo aplicamos el efecto cuando la sección está **dentro de la ventana visible** (entre `offsetTop - outerHeight` y `offsetTop + offsetHeight`). Esto optimiza el rendimiento.
- Dividimos el desplazamiento entre 1.5 para que el fondo se mueva más lento que el scroll, creando el efecto parallax.

Técnica: Material Design en formularios

¿Qué es? El diseño **Material Design** propone que las etiquetas de los campos floten hacia arriba cuando el usuario empieza a escribir, ahorrando espacio y dando feedback visual inmediato.

¿Por qué la aplicamos? Mejora la estética del formulario y guía al usuario mostrando qué campo está activo.

Código JavaScript

```
var propFormulario = {
    formulario: document.formulario_contacto,
    elementos: document.formulario_contacto.elements
};
```

```

};

var metFormulario = {
  inicio: function () {
    for (var i = 0; i < propFormulario.elementos.length; i++) {
      if (propFormulario.elementos[i].type == 'text'
          || propFormulario.elementos[i].type == 'email'
          || propFormulario.elementos[i].nodeName.toLowerCase() == 'textarea') {

        propFormulario.elementos[i].addEventListener('focus', metFormulario.focusInput);
        propFormulario.elementos[i].addEventListener('blur', metFormulario.blurInput);
      }
    }
  },

  focusInput: function () {
    this.parentElement.children[1].className = 'label active';
  },

  blurInput: function () {
    if (this.value == '') {
      this.parentElement.children[1].className = 'label';
    }
  }
};

metFormulario.inicio();

```

Decisiones de implementación:

- `document.formulario_contacto` accede al formulario por su atributo `name`. Es una forma antigua pero válida de acceso en JavaScript.
- Recorremos todos los elementos del formulario y filtramos solo los inputs de texto, email y textarea.
- `focusInput` agrega la clase `.active` al label (segundo hijo del `.input-group`), que lo desplaza hacia arriba mediante CSS.
- `blurInput` revisa si el campo quedó vacío (`this.value == ''`). Si está vacío, devuelve el label a su posición original quitando `.active`.

Técnica: Validación de formulario

¿Qué es? Antes de enviar un formulario, verificamos que los campos obligatorios no estén vacíos. Si lo están, mostramos un mensaje de error y evitamos el envío.

¿Por qué la aplicamos? Evita que lleguen mensajes incompletos al servidor y mejora la experiencia del usuario informando el problema inmediatamente.

Código JavaScript (versión final)

```

(function(){
  var propFormulario = {
    formulario: document.formulario_contacto,
    elementos: document.formulario_contacto.elements,
    error: null,
    textoError: null
  };

  var metFormulario = {
    inicio: function () {
      // ... eventos focus/blur de Material Design ...

```

```

        propFormulario.formulario.addEventListener('submit', metFormulario.validarInputs);
    },

    validarInputs: function (e) {
        for (var i = 0; i < propFormulario.elementos.length; i++) {
            if (propFormulario.elementos[i].value == '') {
                e.preventDefault();

                // Evitamos mensajes duplicados
                if (propFormulario.elementos[i].parentElement.children.length < 3) {
                    propFormulario.error = document.createElement('p');
                    propFormulario.textoError = document.createTextNode(
                        'Por favor llena el campo con tu ' + propFormulario.elementos[i].name);
                    propFormulario.error.appendChild(propFormulario.textoError);
                    propFormulario.error.className = 'error';

                    propFormulario.elementos[i].parentElement.appendChild(propFormulario.error);
                }
            } else {
                // Si el campo ya tiene valor, removemos el error si existe
                if (propFormulario.elementos[i].parentElement.children.length >= 3) {
                    propFormulario.error = propFormulario.elementos[i].parentElement.getElementsByTagName('p')[0];
                    propFormulario.elementos[i].parentElement.removeChild(propFormulario.error);
                }
            }
        }
    }
};

metFormulario.inicio();
})();

```

Decisiones de implementación:

- e.preventDefault() dentro del evento submit detiene el envío real del formulario si hay errores.
- Creamos dinámicamente un <p class="error"> con document.createElement y lo insertamos dentro del .input-group correspondiente.
- Verificamos parentElement.children.length < 3 para no crear múltiples mensajes de error si el usuario intenta enviar varias veces seguidas.
- Si el usuario corrige el campo (tiene valor), removemos el mensaje de error del DOM.

Técnica: Scroll suave

¿Qué es? El **scroll suave** reemplaza el salto brusco entre secciones por un desplazamiento animado y gradual.

¿Por qué la aplicamos? Los enlaces del menú apuntan a secciones de la misma página (href="#portada", etc.). El scroll suave mejora la navegación y da una sensación de fluidez.

Código JavaScript (versión final)

```

var propScroll = {
    posicion: window.pageYOffset,
    scroll_suave: document.getElementsByClassName('scroll-suave'),
    volver_arriba: document.getElementsByClassName('volver-arriba'),
    destino: null,
    seccion_distancia: null,
    intervalo: null
};

```

```

var metScroll = {
  inicio: function () {
    for (var i = 0; i < propScroll.scroll_suave.length; i++) {
      propScroll.scroll_suave[i].addEventListener('click', metScroll.moverse);
    }
    for (var i = 0; i < propScroll.volver_arriba.length; i++) {
      propScroll.volver_arriba[i].addEventListener('click', metScroll.subir);
    }
  },

  moverse: function (e) {
    e.preventDefault();
    clearInterval(propScroll.intervalo);
    propScroll.destino = this.getAttribute('href');
    propScroll.seccion_distancia = document.querySelector(propScroll.destino).offsetTop - 94;

    propScroll.posicion = window.pageYOffset;
    propScroll.intervalo = setInterval(function () {
      if (propScroll.posicion < propScroll.seccion_distancia) {
        propScroll.posicion += 30;
        if (propScroll.posicion >= propScroll.seccion_distancia) {
          clearInterval(propScroll.intervalo);
        }
      } else {
        propScroll.posicion -= 30;
        if (propScroll.posicion <= propScroll.seccion_distancia) {
          clearInterval(propScroll.intervalo);
        }
      }
      window.scrollTo(0, propScroll.posicion);
    }, 15);
  },

  subir: function (e) {
    e.preventDefault();
    clearInterval(propScroll.intervalo);
    propScroll.posicion = window.pageYOffset;
    propScroll.intervalo = setInterval(function(){
      if (propScroll.posicion > 0) {
        propScroll.posicion -= 30;
        if (propScroll.posicion <= 0) {
          clearInterval(propScroll.intervalo);
        }
      } else {
        return;
      }
      window.scrollTo(0, propScroll.posicion);
    }, 15);
  }
};

metScroll.inicio();

```

Decisiones de implementación:

- Usamos `setInterval` para crear una animación manual: cada 15 milisegundos movemos la página 30 píxeles hacia arriba o hacia abajo, según la dirección necesaria.
- `document.querySelector(propScroll.destino).offsetTop - 94`: calculamos la posición exacta de la sección destino y restamos 94px (la altura del header fijo), para que la sección no quede tapada por el menú.
- `clearInterval(propScroll.intervalo)` es fundamental: si el usuario hace clic en otro enlace antes de que termine la animación

anterior, cancelamos el intervalo previo para evitar comportamientos erráticos.

Técnica: Menú móvil deslizable

¿Qué es? Controlamos la apertura y cierre del panel lateral (.slideMenu) mediante JavaScript, agregando o quitando la clase .active.

Código JavaScript

```
var propMenu = {
  burger_menu: document.getElementById('burger_menu'),
  slideMenu: document.getElementById('slideMenu'),
  menu_activo: false,
  elem_menu: document.querySelectorAll('#slideMenu .menu-principal a')
};

var metMenu = {
  inicio: function () {
    propMenu.burger_menu.addEventListener('click', metMenu.toggleMenu);

    for (var i = 0; i < propMenu.elem_menu.length; i++) {
      propMenu.elem_menu[i].addEventListener('click', metMenu.ocultarMenu);
    }
  },

  toggleMenu: function () {
    if (propMenu.menu_activo == false) {
      propMenu.menu_activo = true;
      propMenu.slideMenu.className = propMenu.slideMenu.className + ' active';
    } else {
      propMenu.menu_activo = false;
      propMenu.slideMenu.className = propMenu.slideMenu.className.replace('active', '');
    }
  },

  ocultarMenu: function () {
    propMenu.menu_activo = false;
    propMenu.slideMenu.className = propMenu.slideMenu.className.replace('active', '');
  }
};

metMenu.inicio();
```

Decisiones de implementación:

- propMenu.menu_activo es una bandera booleana que registra si el menú está abierto o cerrado. Esto evita depender únicamente del estado del DOM.
 - className = className + ' active' agrega la clase. Usamos concatenación de strings en lugar de classList para mantener compatibilidad con la sintaxis enseñada en clase.
 - replace('active', '') la quita. Notá que deja un espacio extra en la cadena, pero eso no afecta el funcionamiento.
 - Cuando el usuario toca un enlace del menú móvil (elem_menu), llamamos a ocultarMenu para que el panel se cierre automáticamente y el usuario pueda ver la sección destino.
-

9. Arquitectura final: decisiones clave

Decisión	Justificación
Variables CSS en :root	Centralizan colores y fuentes. Cambiar un valor impacta en todo el sitio sin editar múltiples reglas.
Flexbox para layout	Es más sencillo y moderno que flotadores (float). Permite alinear elementos horizontal y verticalmente con pocas propiedades.
Imágenes como background-image	Permiten que el overlay y el texto se superpongan naturalmente. También facilitan el efecto Lightbox al poder extraer la URL con getComputedStyle.
Header fijo (position: fixed)	El menú siempre está disponible. El margin-top en .main compensa la altura ocupada.
Separación Propiedades/Métodos en JS	Convención didáctica para distinguir “qué datos tengo” de “qué hago con ellos”. Prepara el terreno para entender objetos y clases en futuras materias.
IIFE en scripts	Encapsulan variables y funciones, evitando conflictos con otros scripts y manteniendo el ámbito global limpio.
Media Queries con max-width	Estrategia “Desktop First”: diseñamos para pantallas grandes y ajustamos hacia abajo. Es intuitivo para quienes empiezan porque primero ven el sitio completo en su monitor.

10. Checklist de aprendizaje

Al terminar este proyecto, deberías ser capaz de:

- ☐ Explicar qué es box-sizing: border-box y por qué se usa al inicio de todo proyecto.
- ☐ Crear un layout de dos columnas con Flexbox (display: flex, justify-content, align-items).
- ☐ Definir y usar **variables CSS** (:root y var()).
- ☐ Maquetar un menú de navegación fijo que no tape el contenido al hacer scroll.
- ☐ Escribir una **Media Query** que cambie el layout en pantallas pequeñas.
- ☐ Implementar un menú hamburguesa con un panel deslizable controlado por clases CSS.
- ☐ Explicar cómo funciona un slider infinito usando margin-left, appendChild y setInterval.
- ☐ Crear un sistema de Tabs que muestre/oculte contenido usando className y e.preventDefault().
- ☐ Construir un Lightbox que cree elementos dinámicamente (createElement, appendChild, removeChild).
- ☐ Implementar el efecto **Parallax** con pageYOffset y backgroundPositionY.
- ☐ Aplicar el patrón **Material Design** a un formulario usando eventos focus y blur.
- ☐ Validar un formulario con JavaScript antes de su envío (e.preventDefault(), creación dinámica de mensajes de error).
- ☐ Programar un **scroll suave animado** con setInterval, scrollTo y cálculo de distancias (offsetTop).
- ☐ Entender la diferencia entre getElementsByClassName, getElementById, querySelector y querySelectorAll.

Nota del profesor: Este proyecto no es solo una receta de estilos y scripts. Es un recorrido por decisiones: ¿por qué Flexbox y no tablas? ¿Por qué crear el lightbox con JS y no escribirlo en el HTML? Cada respuesta construye el criterio técnico que se evalúa en la materia. No copies sin preguntar “¿por qué?”. Esa pregunta es la verdadera entrega.